

1. Sintaxă C# vs Java

Acesta este fundamentul. C# derivă din C++ și Java, deci multe lucruri sunt similare, dar există diferențe cheie de sintaxă și convenții.

Diferențe Principale:

- **Metoda Main:**
 - În C#, metoda Main începe cu literă mare (PascalCase).
 - Poate fi void sau poate returna int.
 - Parametrul string[] args este opțional.
- **Afișare la consolă:**
 - Java: `System.out.println("text");`
 - C#: `Console.WriteLine("text");`.
- **Moștenire:**
 - În Java folosești `extends` (clase) și `implements` (interfețe).
 - În C# se folosește doar două puncte : pentru ambele.
- **Tablouri (Arrays):**
 - În C# parantezele drepte sunt mereu după tip, nu după variabilă. Nu poți scrie `int sir[]` ca în Java/C++, ci doar `int[] sir`.
 - Exemplu inițializare: `int[] a = {1, 2, 3};`.
- **Cuvinte cheie:**
 - `const` în C# vs `final` în Java pentru constante.
 - `sealed` în C# vs `final` în Java pentru a preveni moștenirea.
 - `base` în C# vs `super` în Java pentru a accesa clasa părinte.
 - `bool` în loc de `boolean`.

2. Clase și Interfețe în C#

Clase:

- Sunt tipuri de referință (alocate pe heap).
- O clasă poate moșteni o singură clasă de bază, dar poate implementa mai multe interfețe.
- **Destructor:** C# are destructori (similari cu C++), care încep cu `~NumeClasa()`. Sunt apelați automat de Garbage Collector.

Modificatori de acces (Specifici): C# are câțiva modificatori diferiți față de Java:

- **internal:** Accesibil doar din același proiect (assembly).
- **protected internal:** Accesibil din același proiect SAU din subclase (chiar dacă sunt în alt proiect).

Metode Virtuale (Polimorfism): Spre deosebire de Java, unde metodele sunt virtuale implicit:

- În C#, trebuie să marchezi explicit o metodă cu `virtual` dacă vrei să poată fi suprascrisă.
- În clasa derivată, trebuie să folosești explicit `override` pentru a o suprascrie.

1

3. Proprietăți (Properties)

Aceasta este una dintre cele mai mari diferențe de "confort" față de Java. În loc să scrii manual metode `getVariable()` și `setVariable()`, C# are o sintaxă dedicată.

- Proprietățile par a fi câmpuri din exterior, dar se comportă ca metode în interior.
- **Sintaxa:** `public string Nume { get; set; }`
- **Comparație:**
 - **C#:** `p.Name = "Andu";` (Set) sau `string n = p.Name;` (Get).
 - **Java:** `p.setName("Andu");` sau `p.getName();`.

4. Diferențe Majore Java - C# (Funcționalități Extra)

Acestea sunt elemente specifice C# menționate în curs care nu există în Java (sau funcționează diferit):

- **Ref și Out (Transmitere prin referință):**
 - **ref:** Permite modificarea variabilei originale. Variabila trebuie inițializată înainte de apel.
 - **out:** Similar cu `ref`, dar variabila NU trebuie inițializată înainte. Metoda este obligată să îi dea o valoare înainte să se termine.
- **Structuri (struct):**
 - Sunt tipuri valoare (alocate pe stivă), spre deosebire de clase (heap). Nu suportă moștenire.
- **Operatorii `as` și `is`:**
 - **is:** Verifică dacă un obiect este de un anumit tip (returnează `true/false`).
 - **as:** Încearcă să facă conversia (cast). Dacă nu reușește, returnează `null` (nu aruncă excepție).

- **Params:**
 - Permite un număr variabil de parametri (similar cu ... din Java).
- **Indexatori:**
 - Permit accesarea unui obiect ca un vector, folosind [] (ex: obiect[0]).

1. Delegates (Delegați)

Acesta este un concept fundamental în C#, esențial pentru programarea asincronă și evenimente.

- **Definiție:** Un delegat este un **tip de referință** care încapsulează o metodă cu o anumită semnătură (antet).
- **Rol:** Rezolvă problema execuției unei acțiuni atunci când nu știm dinainte care metodă anume sau ce obiect va fi utilizat.
- **Declarare:**

C#

delegate <return type> DelegateName(<list of parameters>);

- Exemplu: `delegate int ArithmeticOperation(int a, int b);`
- **Utilizare:** Orice metodă care respectă semnătura delegatului poate fi atribuită acestuia.
- **Multicast Delegates:** Un delegat poate păstra referințe către mai multe metode. Când este apelat, execută toate metodele înlănțuite. Se folosește operatorul += pentru a adăuga metode.

2. LINQ (Language Integrated Query)

LINQ uniformizează modul în care interogăm datele, indiferent de sursă (colecții, baze de date, XML).

- **Definiție:** LINQ reprezintă o colecție de **metode de extensie** (extension methods) pentru clasele care implementează interfețele IEnumerable sau IQueryable.
- **Sintaxă:** Există două moduri de a scrie interogări:
 1. **Query Syntax (Sintaxa de interogare):** Similară cu SQL (from, where, select).
 2. **Method Syntax (Sintaxa de metodă):** Folosește metodele de extensie și expresii lambda (ex: `.Where(...)`).

3. Operatorii LINQ specificați

În curs sunt listați operatorii disponibili în API-ul System.Linq.Enumerable. Iată detaliile pentru cei ceruți:

- **Where (Filtrare):**
 - Filtrează o secvență de valori pe baza unui predicat (o condiție).
 - *Exemplu Method Syntax:* `stringList.Where(s => s.Contains("Tutorials"))`.

- *Exemplu Query Syntax:*

C#

```
from s in stringList
```

```
where s.Contains("Tutorials")
```

```
select s;
```

.

- **Select (Proiecție):**

- Este utilizat pentru a selecta forma rezultatului (proiecție).
- În exemplele din curs, apare explicit în sintaxa de interogare ca `select s` sau `select p`.

- **Any (Cuantificator):**

- Verifică dacă există cel puțin un element în colecție care satisface o condiție (sau dacă colecția nu este goală).
- Apare în lista de metode API din curs.

- **Aggregate (Agregare):**

- Realizează o operație cumulativă pe o secvență (similar cu `reduce` din alte limbaje).
- Este listat explicit în API-ul `Enumerable` și `Queryable`.

- **Concat (Concatenare):**

- Unește două secvențe într-una singură.
- Apare în lista de extensii disponibile.

- **SelectMany & ElementAtOrDefault:**

- *Notă pentru document:* Acești doi operatori nu apar explicit în listele sau exemplele din paginile cursului `Curs11.pdf`, dar fac parte din funcționalitatea bogată ("Rich Functionality") a colecțiilor menționată.
- Pentru completitudinea notițelor tale (fiind concepte standard):
 - **SelectMany:** "Aplatizează" colecții (ex: dintr-o listă de propoziții, scoate o listă unică de cuvinte).
 - **ElementAtOrDefault:** Returnează elementul de la un index specificat sau valoarea implicită (`null/0`) dacă indexul este în afara limitelor (evitând excepțiile).